

Code Generation in Compiler Design - Interview Guide

Code generation is the compiler phase that turns an intermediate representation (IR) into target-machine code. In interviews, this topic tests whether you can connect abstract compiler theory with real hardware constraints: registers, memory, instruction sets, stack frames, pipelines, and runtime performance.

1. Instruction Selection

1. Overview

Instruction selection is the compiler task of choosing target-machine instructions for each IR operation.

- **Definition:** Mapping IR operations like $x = y + z$ into machine instructions such as `ADD R1, R2, R3`.
- **Why it matters:** Different instruction choices can produce the same result but have different speed, size, and register pressure.
- **Where used:** C/C++ compilers, JVM JITs, JavaScript engines, database query JITs, embedded compilers.
- **Why interviewers ask:** It checks whether you understand that code generation is not just translation; it is optimization under hardware constraints.

2. Core Idea

The compiler has a tree or graph of operations. Instruction selection covers that structure with available machine instructions.

Intuition: If IR says "multiply by 2", the compiler may choose `MUL`, but on many machines `ADD x, x` or `SHL x, 1` is faster or smaller.

Analogy: Translating a sentence into another language. Multiple translations are correct, but one may sound natural and efficient.

Small example:

```
IR:
t1 = b + c
a  = t1

Possible target:
LOAD R1, b
LOAD R2, c
ADD  R3, R1, R2
STORE a, R3
```

Step-by-step:

1. Read the IR operation.
2. Check available target instructions.
3. Pick instructions that preserve meaning.

4. Consider cost: cycles, code size, registers, memory access.
5. Emit instruction sequence.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Tree covering	Match IR expression trees with instruction patterns	Produces compact code	<code>a[i]</code> maps to indexed load	Explain pattern matching
Cost model	Assign cost to instruction choices	Avoids slow choices	<code>SHL</code> cheaper than <code>MUL</code> by 2	Speed vs size tradeoff
Machine idioms	Special instructions for common cases	Exploits hardware	<code>INC x</code> instead of <code>ADD x, 1</code>	Architecture awareness
Complex instructions	One instruction may do many operations	Reduces instruction count	x86 memory operand in <code>ADD</code>	RISC vs CISC
DAG selection	Handles shared subexpressions	Avoids recomputation	<code>b+c</code> used twice	Tree vs DAG complexity

4. Real-World Example

In a browser JavaScript JIT, expression `arr[i] + 1` may become:

```
Bounds check i
Load arr base
Load arr[i] using base + i * element_size
Add immediate 1
```

The instruction selector chooses addressing-mode instructions that combine base, index, scale, and displacement on x86-like CPUs.

5. Diagrams / Mental Models

IR expression tree:



Instruction selection:

```

MUL R1, a, b
ADD R2, R1, c
  
```

6. Common Interview Questions

1. What is instruction selection?

It maps IR operations to target instructions. Expected: mention correctness, cost, target ISA. Mistake: saying it only converts syntax to assembly.

2. Why can the same IR produce different machine code?

The ISA may offer many equivalent instruction sequences. Expected: cost, speed, code size. Mistake: assuming one IR operation equals one instruction.

3. What is a cost model?

A way to estimate instruction cost. Expected: latency, throughput, size, register use. Mistake: counting only instruction number.

4. How does RISC affect instruction selection?

RISC usually needs simpler load/store sequences. Expected: arithmetic on registers, explicit loads/stores. Mistake: saying RISC is always faster.

5. How does CISC affect instruction selection?

CISC can combine memory access and computation. Expected: complex addressing and multi-operation instructions. Mistake: assuming fewer instructions always means faster.

6. What is tree-pattern matching?

Matching IR tree shapes to instruction templates. Expected: bottom-up matching. Mistake: ignoring shared subexpressions.

7. What is maximal munch?

A greedy method that selects the largest matching instruction pattern. Expected: simple but not always optimal. Mistake: calling it globally optimal.

8. What is the role of machine description?

It describes target instructions, operands, and costs. Expected: used by code generator. Mistake: confusing it with source grammar.

9. Why is instruction selection hard for DAGs?

Shared values create reuse choices. Expected: avoid duplication. Mistake: treating DAG as independent

trees.

10. Can instruction selection affect register allocation?

Yes. Some choices use more temporaries. Expected: register pressure link. Mistake: treating phases as independent.

7. Deep-Dive Questions

1. Why is optimal instruction selection difficult?

Because it is a combinatorial optimization problem with interacting costs, sharing, and target constraints.

2. How does dynamic programming help tree instruction selection?

It computes the cheapest pattern cover for each subtree and combines local optimal costs bottom-up.

3. When can greedy selection fail?

When a locally large instruction pattern blocks a globally cheaper sequence.

4. How do vector instructions affect selection?

The compiler must recognize scalar operations that can be packed into SIMD instructions.

5. How do peephole optimizations relate?

They clean small inefficient instruction sequences after selection.

8. Comparison Tables

Tree-Based Selection	DAG-Based Selection
Easier to implement	More complex
Good for expression trees	Handles shared subexpressions
May duplicate work	Can preserve reuse
Dynamic programming works well	Needs more global reasoning
RISC Instruction Selection	CISC Instruction Selection
Simple instructions	Rich instructions
Explicit loads/stores	Memory operands common
More instructions often emitted	Fewer but complex instructions
Easier scheduling	More instruction-specific constraints

9. Common Mistakes

- Assuming every IR instruction maps to exactly one machine instruction.
- Ignoring memory addressing costs.
- Choosing shortest assembly instead of fastest assembly.
- Forgetting that target architecture controls available choices.
- Treating instruction selection, register allocation, and scheduling as unrelated.

10. Edge Cases / Special Cases

- Division by constants may be replaced by multiplication and shifts.
- Some instructions require fixed registers.
- Floating-point instructions may have different precision rules.
- Condition codes can be reused or accidentally overwritten.
- Undefined behavior in source languages can allow surprising instruction choices.

11. How to Explain in Interview

"Instruction selection is the compiler phase that maps IR operations to actual target instructions. It must preserve semantics while choosing efficient sequences based on the target ISA, cost model, addressing modes, and register pressure."

12. Quick Revision Notes

- Key definition: IR-to-machine-instruction mapping.
- Important point: many correct sequences may exist.
- Common comparison: RISC uses simpler instructions; CISC may combine operations.
- Must remember: selection affects register pressure and scheduling.
- Trap: one IR instruction is not always one machine instruction.

13. Practice Tasks

- Convert $a = b + c * d$ into simple three-address code and assembly.
- Compare using `MUL x, 2` vs `SHL x, 1`.
- Write instruction templates for `ADD`, `LOAD`, `STORE`, and indexed load.
- Trace maximal munch on a small expression tree.
- Identify which instruction sequence uses fewer registers.

14. Final Cheat Sheet

- **Core definition:** Select target instructions for IR operations.
- **Why it matters:** Controls speed, code size, and hardware usage.
- **Most asked:** cost model, RISC vs CISC, tree covering, maximal munch.
- **Common comparison:** tree selection vs DAG selection.
- **One-line answer:** "Instruction selection chooses the best target instructions that implement IR semantics under ISA and cost constraints."

2. Register Allocation

1. Overview

Register allocation decides which program values live in CPU registers and which must live in memory.

- **Definition:** Assigning variables and temporary values to a limited number of machine registers.
- **Why it matters:** Register access is much faster than memory access.
- **Where used:** Almost every optimizing compiler and JIT.
- **Why interviewers ask:** It connects liveness analysis, graph coloring, performance, and hardware limits.

2. Core Idea

Programs often have more live values than available registers. The compiler must keep the most useful values in registers and move others to memory.

Analogy: A chef has limited counter space. Frequently used ingredients stay on the counter; rarely used ones go back to the shelf.

Small example:

```
t1 = a + b
t2 = c + d
t3 = t1 * t2
```

If only two registers exist, **t1** may need to stay while computing **t2**, or one value may be temporarily stored in memory.

Step-by-step:

1. Find where each value is live.
2. Build conflicts between simultaneously live values.
3. Assign registers to non-conflicting values.
4. Spill values if registers are insufficient.
5. Insert loads and stores for spilled values.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Liveness analysis	Finds values needed in future	Determines conflicts	x live until last use	Ask live-in/live-out
Live range	Instruction span where value is live	Drives register lifetime	t1 from definition to use	Long ranges cause pressure
Interference graph	Nodes conflict if live together	Basis for graph coloring	a and b both live	Explain edge meaning
Coalescing	Remove unnecessary moves	Faster code	MOV r1,r2 removed	Tradeoff with coloring
Spill choice	Pick value to store in memory	Minimizes slowdown	Spill rarely used value	Heuristics

4. Real-World Example

In a backend server compiled with an optimizing C++ compiler, a hot loop might keep loop index, array pointer, accumulator, and limit in registers. If too many values are live, the compiler spills some values to the stack, increasing memory traffic and slowing the loop.

5. Diagrams / Mental Models

Live ranges:

```
instr:  1  2  3  4  5
a:      [-----]
b:          [-----]
c:              [-----]
```

If a and b overlap, they cannot share a register.

6. Common Interview Questions

1. What is register allocation?

Assigning program values to limited CPU registers. Expected: mention spilling. Mistake: saying it allocates memory.

2. Why are registers important?

They are the fastest storage directly used by CPU instructions. Expected: latency and performance. Mistake: saying memory and registers are similar.

3. What is liveness?

A value is live if it may be used later. Expected: future use. Mistake: saying live means currently assigned.

4. What is a live range?

The interval from definition to last use. Expected: overlap matters. Mistake: ignoring control flow.

5. What is register pressure?

Number of simultaneously live values. Expected: high pressure causes spills. Mistake: counting all variables in function.

6. What is spilling?

Storing a value in memory because no register is available. Expected: loads/stores inserted. Mistake: thinking it deletes the value.

7. Can two variables share one register?

Yes, if their live ranges do not overlap. Expected: reuse registers. Mistake: one variable always needs one fixed register.

8. What is an interference graph?

A graph where edges connect values live at the same time. Expected: coloring analogy. Mistake: connecting data dependencies only.

9. What is coalescing?

Merging move-related values to remove copies. Expected: may increase pressure. Mistake: always beneficial.

10. Why is register allocation hard?

It resembles graph coloring, which is NP-complete in general. Expected: heuristics. Mistake: claiming compilers solve optimally always.

7. Deep-Dive Questions

1. How does control flow affect liveness?

A value is live if it can be used along any future path, so branches require data-flow fixed-point analysis.

2. Why might a compiler split live ranges?

To reduce pressure by keeping a value in registers only around important uses.

3. What are caller-saved and callee-saved registers?

Caller-saved may be clobbered by calls; callee-saved must be restored by the called function.

4. How does allocation differ in a JIT?

JITs often prefer faster algorithms like linear scan because compile time matters.

5. How can instruction selection constrain allocation?

Some instructions require specific registers or register classes.

8. Comparison Tables

Local Allocation	Global Allocation
Within basic block	Across whole function
Simpler	More effective
Misses cross-block reuse	Uses liveness over CFG
Good for simple compilers	Used by optimizing compilers
Graph Coloring	Linear Scan
Better allocation quality	Faster compile time
More complex	Simpler
Common in ahead-of-time compilers	Common in JITs
Uses interference graph	Uses live intervals

9. Common Mistakes

- Confusing register allocation with memory allocation.
- Forgetting that values can share registers if lifetimes do not overlap.
- Ignoring function calls and calling conventions.
- Assuming spilling always happens only once.
- Treating liveness as purely linear, ignoring branches.

10. Edge Cases / Special Cases

- Fixed-register instructions like division on some architectures.

- Different register classes: integer, floating-point, vector.
- Values live across function calls need special handling.
- Exception paths can extend live ranges.
- Debug builds may allocate poorly to preserve debuggability.

11. How to Explain in Interview

"Register allocation assigns live program values to a limited set of CPU registers. If too many values are live at once, the compiler spills some to memory. Good allocation reduces memory traffic and improves performance."

12. Quick Revision Notes

- Key definition: mapping live values to registers.
- Important point: overlap means conflict.
- Common comparison: graph coloring vs linear scan.
- Must remember: spills add loads/stores.
- Trap: data dependency is not the same as interference.

13. Practice Tasks

- Draw live ranges for three-address code.
- Build an interference graph for a small basic block.
- Color a graph with three registers.
- Choose a spill candidate and justify it.
- Compare graph coloring and linear scan on a loop.

14. Final Cheat Sheet

- **Core definition:** Assign variables/temporaries to machine registers.
- **Why it matters:** Registers are faster than memory.
- **Most asked:** liveness, interference graph, spilling, graph coloring.
- **Common comparison:** graph coloring vs linear scan.
- **One-line answer:** "Register allocation keeps frequently needed live values in registers and spills the rest when registers run out."

3. Register Spilling

1. Overview

Register spilling happens when the compiler cannot keep every live value in registers and must store some values in memory.

- **Definition:** Moving a value from register to memory, usually the stack, and loading it back later.
- **Why it matters:** Spills are expensive because memory is slower than registers.
- **Where used:** Optimizing compilers, JITs, embedded compilers with few registers.
- **Why interviewers ask:** It tests practical understanding of register pressure and performance regressions.

2. Core Idea

When active values exceed available registers, the compiler selects a victim value to store in memory.

Analogy: You have two hands but three objects. One object must be put on the table temporarily.

Small example:

Only 2 registers: R1, R2

t1 = a + b -> R1

t2 = c + d -> R2

t3 = e + f -> need register

spill t1 to stack

compute t3 in R1

reload t1 when needed

Step-by-step:

1. Detect register shortage.
2. Select spill candidate.
3. Store candidate to stack slot.
4. Use freed register.
5. Reload spilled value before later use.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Spill cost	Estimated runtime penalty	Pick cheapest victim	Spill value used once later	Cost model
Stack slot	Memory location for spilled value	Preserves value	[fp-8]	Stack frame impact
Rematerialization	Recompute instead of reload	Can be cheaper	constant 0 or address	Avoid memory
Live-range splitting	Spill only part of lifetime	Reduces overhead	keep in reg in hot loop	Advanced allocation
Spill code	Inserted load/store instructions	Affects performance	STORE, LOAD	Correct placement

4. Real-World Example

A database query engine JIT may compile a predicate with many columns. If it keeps too many column values live while evaluating expressions, the allocator spills some to stack. This can make a query slower even though the high-level logic is unchanged.

5. Diagrams / Mental Models

Before spill:

R1: a
R2: b
Need: c

After spill:

stack[slot0] = a
R1: c
R2: b

Before using a:
R1 = stack[slot0]

6. Common Interview Questions

1. What is register spilling?

Storing register values in memory when registers are insufficient. Expected: reload later. Mistake: saying value is lost.

2. Why is spilling expensive?

It adds memory load/store instructions. Expected: memory latency. Mistake: counting only extra instructions.

3. Where are spilled values stored?

Usually in stack-frame slots. Expected: stack memory. Mistake: saying heap always.

4. When does spilling occur?

When live values exceed available registers or constraints block allocation. Expected: register pressure. Mistake: only large programs spill.

5. How does compiler choose what to spill?

It estimates cost using use frequency, loop depth, and live range. Expected: spill cheap values. Mistake: random choice.

6. What is rematerialization?

Recomputing a value instead of loading it. Expected: useful for constants/simple addresses. Mistake: using it for expensive expressions.

7. Can spilling happen inside loops?

Yes, but it is costly. Expected: avoid hot-loop spills if possible. Mistake: treating all spills equally.

8. What is spill code?

Loads and stores inserted to save/recover spilled values. Expected: correct placement around uses/defs. Mistake: insert at function start only.

9. How does spilling affect stack frame size?

It adds stack slots. Expected: larger frame. Mistake: no memory impact.

10. Can spilled variables still be optimized?

Yes, with live-range splitting, rematerialization, and better scheduling. Expected: mitigation. Mistake: spill means permanently memory-resident.

7. Deep-Dive Questions

1. Why is spilling in inner loops worse?

Loads/stores repeat many times, multiplying cost by loop iteration count.

2. How does loop depth affect spill cost?

Uses in deeper loops receive higher spill cost because they execute more often.

3. Can spill insertion create new register pressure?

Yes. Reloads need registers too, sometimes causing cascading spills.

4. What is optimistic coloring?

Temporarily remove a high-degree node hoping neighbors use fewer colors later; spill only if coloring fails.

5. Why might rematerialization be better than reloading?

Recomputing a constant or simple value may be cheaper than memory access.

8. Comparison Tables

Spilling	Rematerialization
Store/load value from memory	Recompute value
Works for any value	Works for cheap recomputable values
Adds memory traffic	Adds compute instructions
Bad in hot loops	Often good for constants
Spill Entire Live Range	Split Live Range
Simpler	More precise
More memory traffic	Less memory traffic
Easy to implement	More allocator complexity
Common in simple compilers	Common in optimizing compilers

9. Common Mistakes

- Believing spilling only happens for source variables, not temporaries.
- Forgetting spilled values need stack storage.
- Ignoring loop frequency when selecting spills.
- Assuming reloads can be placed anywhere.
- Missing that calls can force values to spill.

10. Edge Cases / Special Cases

- A spill slot may be reused by non-overlapping spilled values.
- Some values are better recomputed than spilled.
- Stack alignment can affect spill slot layout.
- Vector spills may require larger aligned slots.
- Security features may clear or protect stack data.

11. How to Explain in Interview

"Register spilling is what the compiler does when there are more live values than registers. It stores less valuable values to stack slots and reloads them later, but this adds memory traffic, so good compilers try to spill rarely used or cold values."

12. Quick Revision Notes

- Key definition: temporary movement from register to memory.
- Important point: spilling is caused by high register pressure.
- Common comparison: spilling vs rematerialization.
- Must remember: hot-loop spills are expensive.
- Trap: spilling does not mean deleting a value.

13. Practice Tasks

- Given four live values and two registers, choose spill candidates.
- Insert spill code into a three-address instruction sequence.
- Calculate spill cost using loop depth.
- Find where reloading is necessary.
- Identify values suitable for rematerialization.

14. Final Cheat Sheet

- **Core definition:** Store register values in memory when registers run out.
 - **Why it matters:** Spills slow code through memory traffic.
 - **Most asked:** spill cost, stack slots, rematerialization.
 - **Common comparison:** spill vs rematerialize.
 - **One-line answer:** "Spilling saves less useful live values to memory so scarce registers can hold more important values."
-

4. Addressing Modes

1. Overview

Addressing modes describe how a machine instruction finds its operands in registers or memory.

- **Definition:** Ways to specify operand locations, such as immediate, register, direct memory, indirect, indexed, or base-plus-offset.
- **Why it matters:** Good addressing-mode use reduces instruction count and improves memory access efficiency.

- **Where used:** Assembly generation, array access, pointer dereference, stack-frame access, object-field access.
- **Why interviewers ask:** It tests hardware awareness and ability to translate high-level memory operations.

2. Core Idea

An instruction needs operands. Addressing modes tell the CPU where those operands are and how to compute their addresses.

Analogy: Giving directions. "Use this exact number", "look in this drawer", or "go to shelf base plus 3 boxes".

Small example:

```
a[i] on a 4-byte int array:  
address = base(a) + i * 4
```

On x86-like machines, this may fit one addressing mode:

```
MOV R1, [BASE + INDEX * 4]
```

Step-by-step:

1. Identify data location.
2. Compute base address.
3. Add offset or scaled index.
4. Emit load/store instruction using supported mode.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Immediate	Operand is inside instruction	Fast constants	ADD R1, 5	No memory read for operand
Register	Operand in register	Fastest common mode	ADD R1, R2	Register allocation link
Direct	Address explicitly named	Simple globals	LOAD R1, [x]	Static storage
Indirect	Register holds address	Pointers	LOAD R1, [R2]	C pointer dereference
Indexed	Base plus index/scale	Arrays	[base+i*4]	Array lowering
Base offset	Base register plus constant	Stack/object fields	[fp-8]	Stack frames

4. Real-World Example

In an operating system kernel, accessing a process table entry may use base-plus-index addressing:

```
process = process_table[pid]
address = table_base + pid * sizeof(process)
```

The compiler uses the addressing mode to generate efficient memory access.

5. Diagrams / Mental Models

```
Address calculation:

base address of array
      +
index * element size
      +
field offset
      =
final memory address
```

6. Common Interview Questions

1. What is an addressing mode?

A way an instruction specifies operand location. Expected: register/memory/immediate. Mistake: confusing with network addressing.

2. What is immediate addressing?

Operand value is encoded in instruction. Expected: constants. Mistake: saying it reads memory immediately.

3. What is register addressing?

Operand is in a CPU register. Expected: fast access. Mistake: saying variable name is a register.

4. What is indirect addressing?

A register or memory location contains the address of operand. Expected: pointer dereference. Mistake: confusing with direct memory.

5. How are arrays accessed?

Base address plus index times element size. Expected: scaling. Mistake: forgetting element size.

6. How are struct fields accessed?

Base address plus fixed field offset. Expected: compile-time offsets. Mistake: scanning fields at runtime.

7. Why are addressing modes important for code generation?

They can combine address computation with load/store. Expected: fewer instructions. Mistake: only syntax matter.

8. How is a local variable addressed?

Usually stack/frame pointer plus offset. Expected: `[fp-offset]` or `[sp+offset]`. Mistake: heap by default.

9. What is base-plus-offset addressing?

Address is register base plus constant displacement. Expected: stack and object access. Mistake: offset must be variable.

10. How do addressing modes differ in RISC and CISC?

CISC often has richer modes; RISC often simpler load/store. Expected: ISA difference. Mistake: one universal model.

7. Deep-Dive Questions

1. Why might complex addressing not always be fastest?

It can increase instruction latency or limit scheduling on some CPUs.

2. How does alignment affect memory addressing?

Misaligned access may be slower or illegal depending on architecture.

3. How are multidimensional arrays addressed?

Row-major: `base + ((i * cols) + j) * element_size`.

4. How do addressing modes support stack frames?

Locals and parameters are accessed using stack/frame pointer offsets.

5. How do object fields map to addressing modes?

Object pointer is base; field offset is displacement.

8. Comparison Tables

Addressing Mode	Operand Source	Example	Common Use
Immediate	Instruction itself	<code>ADD R1, 10</code>	Constants
Register	Register	<code>ADD R1, R2</code>	Fast computation
Direct	Fixed memory address	<code>LOAD R1, [x]</code>	Globals
Indirect	Address in register	<code>LOAD R1, [R2]</code>	Pointers
Indexed	Base + scaled index	<code>[A + i*4]</code>	Arrays
Base offset	Base + constant	<code>[fp-8]</code>	Locals/fields
Simple Addressing		Complex Addressing	
Easier for hardware		More expressive	
More instructions may be needed		Fewer instructions possible	
Common in RISC		Common in CISC	
Easier scheduling		May have extra constraints	

9. Common Mistakes

- Forgetting array index scaling.
- Confusing address with value stored at address.
- Assuming all variables have fixed memory addresses.
- Ignoring stack pointer changes.
- Thinking complex addressing is always better.

10. Edge Cases / Special Cases

- Unaligned access can trap on some machines.
- Stack pointer relative offsets change after pushes/pops.
- Large offsets may not fit in instruction encoding.
- Position-independent code uses relative addressing.
- Garbage-collected runtimes may need barriers around object-field access.

11. How to Explain in Interview

"Addressing modes are the operand-location rules supported by the target ISA. They tell the generated instruction whether to use a constant, register, direct memory address, pointer dereference, or base-plus-index calculation."

12. Quick Revision Notes

- Key definition: how instructions locate operands.
- Important point: arrays use $\text{base} + \text{index} * \text{size}$.
- Common comparison: immediate vs register vs indirect.
- Must remember: local variables often use stack offsets.
- Trap: address and value are different.

13. Practice Tasks

- Translate $x = a[i]$ into address calculation.
- Compute offset for $\text{matrix}[i][j]$ in row-major layout.
- Identify addressing modes in sample assembly.
- Draw stack frame offsets for locals and parameters.
- Compare RISC and CISC code for array access.

14. Final Cheat Sheet

- **Core definition:** Mechanism used by instruction to locate operands.
- **Why it matters:** Efficient addressing reduces loads and arithmetic.
- **Most asked:** immediate, register, indirect, indexed, base offset.
- **Common comparison:** RISC simple modes vs CISC complex modes.
- **One-line answer:** "Addressing modes define how generated instructions compute or find operand addresses."

5. Target Machine Model

1. Overview

The target machine model is the compiler's description of the hardware and ABI it generates code for.

- **Definition:** A model of registers, instruction set, memory layout, data sizes, alignment, calling convention, and execution costs.
- **Why it matters:** Correct code generation depends on machine-specific details.
- **Where used:** Compiler backends, assemblers, linkers, cross-compilers, JITs.
- **Why interviewers ask:** It checks whether you understand why compilers have architecture-specific backends.

2. Core Idea

A compiler cannot generate code in a vacuum. It needs to know what the target CPU can do and how the operating system expects functions and data to behave.

Analogy: Writing instructions for a robot. You must know its tools, hands, speed, and rules before assigning tasks.

Small example:

```
int size:
x86-64 Linux: usually 4 bytes
pointer size: 8 bytes
stack alignment: commonly 16 bytes before calls
```

Step-by-step:

1. Define registers and register classes.
2. Define instruction formats.
3. Define memory alignment and data layout.
4. Define calling convention.
5. Define instruction costs and constraints.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
ISA	Available instructions	Controls selection	ADD, LOAD, JMP	Backend target
Registers	CPU storage locations	Controls allocation	general, FP, vector	Register classes
Data layout	Size/alignment of types	Correct memory access	int=4, pointer=8	Struct layout
Endianness	Byte order in memory	Binary compatibility	little endian x86	Serialization bugs
ABI	OS/toolchain binary rules	Function calls/linking	parameter registers	Calling convention

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Cost model	Estimated instruction cost	Optimization choices	load slower than add	Performance

4. Real-World Example

A cross-compiler targeting ARM from an x86 laptop must generate ARM instructions, use ARM registers, follow ARM ABI rules, and lay out data as the target platform expects, not as the host laptop does.

5. Diagrams / Mental Models

Compiler backend depends on:

Source IR

|

v

Target machine model

|-- registers

|-- instruction set

|-- data layout

|-- ABI/calling convention

|-- costs

v

Machine code

6. Common Interview Questions

1. What is a target machine model?

Compiler's model of target hardware and ABI. Expected: registers, ISA, data layout. Mistake: only CPU name.

2. Why does compiler need target information?

To emit correct and efficient machine code. Expected: machine-specific codegen. Mistake: same assembly for all CPUs.

3. What is ISA?

Instruction Set Architecture: visible instructions and behavior. Expected: programmer/compiler view of CPU. Mistake: implementation microarchitecture.

4. What is ABI?

Binary interface rules for calls, layout, and linking. Expected: calling convention, register use. Mistake: same as API.

5. Why does data layout matter?

Incorrect size/alignment causes wrong memory access. Expected: structs, arrays, padding. Mistake: types are abstract at runtime.

6. What is register class?

Group of registers usable for certain value types. Expected: integer, FP, vector. Mistake: all registers interchangeable.

7. What is endianness?

Byte order for multi-byte values. Expected: little vs big endian. Mistake: bit order.

8. What is alignment?

Address multiple required/preferred for data. Expected: performance/correctness. Mistake: only memory saving.

9. Why do cross-compilers need target triples?

They encode architecture, vendor, OS, ABI. Expected: target selection. Mistake: host machine decides everything.

10. How does cost model affect optimization?

It guides choices among legal instruction sequences. Expected: latency/throughput. Mistake: exact runtime prediction always.

7. Deep-Dive Questions

1. How is target machine model different from microarchitecture?

ISA/ABI define visible behavior; microarchitecture defines internal implementation like pipelines and caches.

2. Why can same ISA have different performance models?

Different CPU generations execute the same instruction with different latencies and throughput.

3. How do vector units affect target model?

They add register classes, instruction constraints, alignment concerns, and width-specific operations.

4. How does OS affect code generation?

OS ABI controls calling convention, executable format, system calls, and TLS access.

5. Why is stack alignment part of target model?

Calls and SIMD instructions may require aligned stack addresses for correctness/performance.

8. Comparison Tables

ISA	ABI
Defines CPU instructions	Defines binary interaction rules
Hardware-facing	OS/toolchain-facing
Example: x86-64, ARM64	Example: System V AMD64, Windows x64
Used for instruction selection	Used for calls, layout, linking
Target Machine	Host Machine
Machine code is generated for it	Compiler runs on it

Target Machine	Host Machine
Determines data layout	Determines compiler execution
Important in cross-compilation	May be unrelated
Example: ARM phone	Example: x86 laptop

9. Common Mistakes

- Confusing host and target machines.
- Treating ABI as the same thing as API.
- Ignoring alignment and padding.
- Assuming all x86-64 platforms use identical calling conventions.
- Forgetting that cost model is approximate.

10. Edge Cases / Special Cases

- Same source type can have different size across targets.
- Some architectures require strict alignment.
- Some targets have no hardware floating point.
- Embedded targets may have very few registers.
- Position-independent code changes addressing choices.

11. How to Explain in Interview

"The target machine model is the backend's knowledge of the CPU and platform: instructions, registers, data sizes, alignment, ABI, and costs. Without it, the compiler cannot generate correct target-specific code."

12. Quick Revision Notes

- Key definition: hardware/platform model used by backend.
- Important point: target, not host, determines generated code.
- Common comparison: ISA vs ABI.
- Must remember: data layout affects correctness.
- Trap: same language code does not imply same binary layout everywhere.

13. Practice Tasks

- Compare 32-bit and 64-bit pointer layouts.
- Draw struct padding for a small C struct.
- List integer, floating-point, and vector register classes.
- Explain target triple components.
- Compare System V AMD64 and Windows x64 argument passing at a high level.

14. Final Cheat Sheet

- **Core definition:** Compiler's model of target hardware and ABI.
- **Why it matters:** Required for correct machine-specific code.
- **Most asked:** ISA, ABI, registers, data layout, alignment.

- **Common comparison:** host vs target, ISA vs ABI.
 - **One-line answer:** "A target machine model tells the compiler what machine it is generating code for and what rules that machine follows."
-

6. Basic Code-Generation Algorithm

1. Overview

The basic code-generation algorithm translates IR into target code while managing registers, memory, and control flow.

- **Definition:** A systematic method for converting intermediate code into assembly or machine code.
- **Why it matters:** It is the foundation before advanced optimizations.
- **Where used:** Educational compilers, simple backends, JIT baseline compilers.
- **Why interviewers ask:** It tests whether you can manually translate IR to assembly.

2. Core Idea

For each IR instruction, generate equivalent target instructions. Keep track of where each value currently lives: register, memory, or both.

Analogy: Executing a recipe with limited bowls. You track which ingredient is in which bowl and when to wash/reuse bowls.

Small example:

```
IR:
t1 = a + b
c  = t1

Target:
LOAD R1, a
ADD  R1, b
STORE c, R1
```

Step-by-step:

1. Partition IR into basic blocks.
2. For each instruction, ensure operands are available.
3. Load operands into registers if needed.
4. Emit target operation.
5. Update register/address descriptors.
6. Store live values before block exits if needed.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Basic block	Straight-line code region	Easier local generation	no internal jumps	Block-level codegen
Next-use info	Whether value is used later	Frees registers early	dead temp can be overwritten	Register choice
Register descriptor	Tracks value in each register	Avoids unnecessary loads	<code>R1 contains x</code>	Codegen bookkeeping
Address descriptor	Tracks where variable value is	Ensures correctness	<code>x in R1 and memory</code>	Store decisions
Control flow	Branch and jump emission	Preserves program logic	<code>ifFalse goto L1</code>	Labels and branches

4. Real-World Example

A simple educational compiler for a C-like language may translate three-address code into assembly one basic block at a time. It may not do global register allocation, but it can still generate correct code by loading operands, computing results, and storing final values.

5. Diagrams / Mental Models

```

IR basic block
|
v
For each statement:
  load operands
  choose register
  emit instruction
  update descriptors
  store if needed
|
v
Target assembly

```

6. Common Interview Questions

1. What is code generation?

Translating IR to target code. Expected: registers, instructions, memory. Mistake: parsing source code.

2. What is a basic block?

Straight-line code with one entry and one exit. Expected: no jumps inside except end. Mistake: any group of lines.

3. Why use basic blocks in code generation?

They simplify local optimization and register tracking. Expected: straight-line reasoning. Mistake: only for syntax.

4. What is a register descriptor?

It records what value each register contains. Expected: bookkeeping. Mistake: hardware register file itself.

5. What is an address descriptor?

It records where current value of a variable is stored. Expected: register/memory locations. Mistake: memory address only.

6. What is next-use information?

It tells whether/when a value will be used later. Expected: helps register reuse. Mistake: next instruction only.

7. How is $x = y + z$ generated?

Load y , add z , store/update x . Expected: operand availability. Mistake: no registers mentioned.

8. When must values be stored to memory?

Before overwriting if memory needs current value, at block exits for live variables, or around calls. Expected: correctness. Mistake: always store after every instruction.

9. How are branches generated?

Evaluate condition and emit conditional/unconditional jumps to labels. Expected: labels. Mistake: source-level `if` remains.

10. Why is naive code generation inefficient?

It may reload/store too often and ignore global liveness. Expected: redundant memory traffic. Mistake: it is always optimal.

7. Deep-Dive Questions**1. How do descriptors reduce loads and stores?**

They tell the compiler when a value is already in a register or memory.

2. Why flush registers at basic block boundaries in simple codegen?

Without global analysis, storing live variables preserves correctness across unknown successor paths.

3. How does next-use info choose registers?

Prefer overwriting registers holding dead values or values whose next use is farthest.

4. How are arrays different from scalar variables?

Array access requires address computation and may alias with other memory accesses.

5. How do function calls complicate basic code generation?

Calls may clobber registers and require argument passing according to calling convention.

8. Comparison Tables

Naive Code Generation	Optimized Code Generation
Simple and correct	More complex
Many loads/stores	Fewer memory operations

Naive Code Generation	Optimized Code Generation
Local decisions	Global analysis
Good for teaching/baseline JIT	Good for production performance
Register Descriptor	Address Descriptor
Register -> values	Variable -> locations
Answers "what is in R1?"	Answers "where is x?"
Helps avoid reloads	Helps know if memory is current
Updated after instructions	Updated after loads/stores/moves

9. Common Mistakes

- Storing every temporary unnecessarily.
- Forgetting to update descriptors after a store.
- Reusing a register while its value is still live.
- Ignoring branch targets and labels.
- Assuming local code generation gives globally optimal code.

10. Edge Cases / Special Cases

- Aliasing can invalidate memory assumptions.
- Function calls may clobber registers.
- Division may require special registers.
- Short-circuit boolean logic affects control flow.
- Values live on multiple successor paths must be preserved.

11. How to Explain in Interview

"A basic code generator walks IR instructions, loads operands into registers, emits target instructions, tracks where values live using descriptors, and stores needed values before control leaves the block."

12. Quick Revision Notes

- Key definition: IR-to-target translation algorithm.
- Important point: descriptors track register/memory state.
- Common comparison: naive vs optimized codegen.
- Must remember: basic blocks simplify local reasoning.
- Trap: do not overwrite live values.

13. Practice Tasks

- Generate assembly for $a = b + c; d = a * e$.
- Maintain register descriptors after each instruction.
- Use next-use info to pick which register to reuse.
- Emit branch code for $\text{if } x < y \text{ goto } L1$.
- Translate a while loop into labels and jumps.

14. Final Cheat Sheet

- **Core definition:** Algorithm for emitting target code from IR.
 - **Why it matters:** Foundation of compiler backend.
 - **Most asked:** basic blocks, descriptors, next-use, branches.
 - **Common comparison:** register descriptor vs address descriptor.
 - **One-line answer:** "Basic code generation emits target instructions from IR while tracking values in registers and memory."
-

7. Graph-Coloring Register Allocation

1. Overview

Graph-coloring register allocation models register assignment as coloring a graph where interfering values need different colors.

- **Definition:** Build an interference graph and color it using available registers as colors.
- **Why it matters:** It gives high-quality global register allocation.
- **Where used:** Optimizing ahead-of-time compilers and some high-tier JITs.
- **Why interviewers ask:** It combines graph theory, liveness analysis, and compiler optimization.

2. Core Idea

If two values are live at the same time, they cannot use the same register. Represent each value as a node. Add an edge between values that interfere. Then color nodes with at most K colors, where K is number of registers.

Analogy: Exam scheduling. Two exams with common students cannot be in the same slot. Colors are time slots.

Small example:

```
Interference:
a conflicts with b and c
b conflicts with a
c conflicts with a

With 2 registers:
a -> R1
b -> R2
c -> R2 (b and c do not conflict)
```

Step-by-step:

1. Run liveness analysis.
2. Build interference graph.
3. Simplify low-degree nodes.
4. Select spill candidates for high-pressure nodes.
5. Assign colors while popping stack.

6. Insert spill code if coloring fails.

3. Important Subtopics

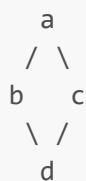
Subtopic	Meaning	Why It Matters	Example	Interview Angle
Interference graph	Conflicting live values	Core structure	edge means no same register	Graph meaning
K-coloring	Assign up to K registers	Hardware limit	K=8 registers	NP-complete idea
Simplification	Remove degree < K nodes	Easier coloring	low conflict value	Stack algorithm
Spill candidate	Node likely impossible/expensive	Handles failure	high-degree low-use node	Heuristic
Coalescing	Merge move-related nodes	Removes MOVs	<code>x=y</code>	Benefit vs pressure
Precolored nodes	Fixed hardware registers	Constraints	return register	ABI constraints

4. Real-World Example

An optimizing C compiler compiling a numeric function may use graph coloring to keep loop accumulators and array pointers in registers while spilling rarely used values outside the loop.

5. Diagrams / Mental Models

Interference graph:



Colors/registers:

R1, R2, R3

Adjacent nodes cannot share color.

6. Common Interview Questions

1. What is graph-coloring register allocation?

Assigning registers by coloring an interference graph. Expected: nodes values, colors registers. Mistake: coloring CFG blocks.

2. What does an edge mean?

Two values are live at the same time. Expected: cannot share register. Mistake: data dependency.

3. What is K in K-coloring?

Number of available registers. Expected: physical registers. Mistake: number of variables.

4. Why is graph coloring hard?

General graph coloring is NP-complete. Expected: compilers use heuristics. Mistake: always exact.

5. What happens if graph cannot be colored?

Spill one or more values and retry. Expected: insert memory loads/stores. Mistake: compilation fails normally.

6. What is simplification?

Remove low-degree nodes because they can be colored later. Expected: degree < K. Mistake: deleting variable.

7. What is coalescing?

Merging move-related nodes to eliminate copies. Expected: may increase degree. Mistake: always safe.

8. What are precolored nodes?

Values fixed to specific machine registers. Expected: ABI/instruction constraints. Mistake: compiler chooses all registers freely.

9. How does liveness feed graph coloring?

Liveness tells which values overlap and need edges. Expected: data-flow analysis. Mistake: graph made from syntax names only.

10. Why spill high-degree nodes?

They constrain coloring heavily, but actual choice also considers use cost. Expected: degree and spill cost. Mistake: always highest degree.

7. Deep-Dive Questions

1. What is Briggs optimistic coloring?

It delays spilling high-degree nodes, hoping they can be colored during select phase.

2. What is George/Appel coalescing intuition?

Coalesce moves only when it is unlikely to make coloring impossible.

3. How do register classes complicate coloring?

A value may only be colorable by a subset of registers, such as FP or vector registers.

4. Why can coalescing hurt?

Merging nodes can increase degree and cause spills.

5. How does live-range splitting improve coloring?

It creates smaller ranges with fewer interferences.

8. Comparison Tables

Graph Coloring		Linear Scan
High-quality allocation		Very fast allocation
Uses interference graph		Uses sorted live intervals
More compile time		Lower compile time
Good for static optimizing compilers		Good for JIT baseline/tiered compilers
Simplify	Spill	Coalesce
Remove easy nodes	Choose memory candidate	Merge move-related nodes
Reduces graph	Handles too many conflicts	Removes copies
Usually safe for degree < K	Adds load/store cost	Can increase pressure

9. Common Mistakes

- Thinking interference edges mean assignment dependencies.
- Forgetting register classes and fixed registers.
- Saying graph coloring never spills.
- Ignoring that coalescing can make allocation worse.
- Treating NP-complete as "impossible"; compilers use good heuristics.

10. Edge Cases / Special Cases

- Precolored return registers.
- Call-clobbered registers around function calls.
- Architecture-specific paired registers.
- Values needing contiguous vector registers.
- Exception handling and debug info can extend liveness.

11. How to Explain in Interview

"Graph-coloring allocation builds a graph where nodes are live values and edges mean two values cannot share a register. Registers are colors. If the graph cannot be colored with available registers, the compiler spills selected values to memory."

12. Quick Revision Notes

- Key definition: register allocation as graph coloring.
- Important point: edge means simultaneous liveness.
- Common comparison: graph coloring vs linear scan.
- Must remember: spilling may require rebuilding graph.
- Trap: dependency graph is not interference graph.

13. Practice Tasks

- Build an interference graph from live ranges.
- Color a graph with two and three registers.

- Choose spill candidates using degree and use frequency.
- Show how coalescing removes a move.
- Explain why a triangle graph needs three colors.

14. Final Cheat Sheet

- **Core definition:** Assign registers by coloring interference graph.
- **Why it matters:** Strong global allocation quality.
- **Most asked:** interference graph, K-coloring, spilling, coalescing.
- **Common comparison:** graph coloring vs linear scan.
- **One-line answer:** "Graph coloring treats registers as colors and assigns them so simultaneously live values get different registers."

8. Instruction Scheduling

1. Overview

Instruction scheduling reorders instructions to improve CPU pipeline usage without changing program meaning.

- **Definition:** Rearranging independent instructions to reduce stalls and improve throughput.
- **Why it matters:** CPUs execute instructions through pipelines; bad order can waste cycles.
- **Where used:** Optimizing compilers, CPU-specific backends, VLIW/EPIC compilers, GPU compilers.
- **Why interviewers ask:** It tests dependency reasoning and hardware-performance awareness.

2. Core Idea

If instruction B depends on instruction A, B must wait for A's result. But unrelated instruction C can run between them to hide latency.

Analogy: While rice cooks, chop vegetables instead of waiting idle.

Small example:

```
Bad:
LOAD R1, [a]      ; load has latency
ADD  R2, R1, 1    ; waits for load
MUL  R3, R4, R5   ; independent
```

```
Better:
LOAD R1, [a]
MUL  R3, R4, R5
ADD  R2, R1, 1
```

Step-by-step:

1. Build dependency graph.
2. Respect true dependencies and side effects.

3. Estimate instruction latencies.
4. Pick ready instructions that reduce stalls.
5. Emit reordered sequence.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Data dependency	One instruction needs another's result	Preserves correctness	ADD after LOAD	RAW dependency
Anti-dependency	Later write must not happen before earlier read	Register reuse issue	read R1 before write R1	WAR dependency
Output dependency	Two writes to same location ordered	Preserve final value	write R1 twice	WAW dependency
Latency	Delay before result ready	Causes stalls	memory load	Pipeline performance
List scheduling	Pick ready instructions from dependency graph	Practical algorithm	ready queue	Common algorithm
Basic-block scheduling	Reorder inside block	Simpler	straight-line code	Local scheduling

4. Real-World Example

In a database engine scanning columns, loads from memory may have high latency. A compiler schedules independent arithmetic and future loads between a load and its use to keep the CPU busy.

5. Diagrams / Mental Models

Dependency graph:

LOAD a ---> ADD

MUL b,c (independent)

Scheduler can place MUL between LOAD and ADD.

6. Common Interview Questions

1. What is instruction scheduling?

Reordering instructions to reduce stalls without changing meaning. Expected: dependencies. Mistake: choosing instructions.

2. Why is scheduling needed?

To hide latency and improve pipeline utilization. Expected: CPU pipeline. Mistake: only reduce code size.

3. What is a data dependency?

One instruction needs another's result. Expected: RAW. Mistake: any same register use.

4. What is RAW dependency?

Read after write; true dependency. Expected: must preserve. Mistake: can be renamed away generally.

5. What is WAR dependency?

Write after read; anti-dependency. Expected: register reuse issue. Mistake: true data flow.

6. What is WAW dependency?

Write after write; output dependency. Expected: preserve final write order. Mistake: no issue if same register.

7. Can all independent instructions be reordered?

Not always; memory aliasing, exceptions, volatile operations, and calls constrain movement. Expected: side effects. Mistake: pure dependency graph only.

8. What is list scheduling?

Scheduling from ready instructions based on priority. Expected: dependency DAG and ready list. Mistake: linked list operation.

9. How does scheduling affect register pressure?

Moving instructions can extend live ranges. Expected: tradeoff. Mistake: scheduling only improves code.

10. What is delay-slot scheduling?

Filling branch delay slots on architectures that execute instruction after branch. Expected: older RISC. Mistake: universal CPU feature.

7. Deep-Dive Questions

1. Why can scheduling before register allocation be risky?

It may increase live ranges and cause more spills.

2. Why schedule after register allocation?

It knows actual registers but must respect artificial dependencies from register reuse.

3. What is software pipelining?

Loop scheduling that overlaps instructions from different iterations.

4. How does memory alias analysis help scheduling?

It proves whether loads/stores can be safely reordered.

5. How do out-of-order CPUs change compiler scheduling?

Hardware can reorder dynamically, but compiler scheduling still helps on constrained or predictable targets.

8. Comparison Tables

Instruction Scheduling	Instruction Selection
Reorders chosen instructions	Chooses which instructions to use

Instruction Scheduling		Instruction Selection
Concerned with latency/pipeline		Concerned with ISA patterns/cost
Must preserve dependencies		Must preserve semantics
Happens before/after allocation		Happens during code generation
RAW	WAR	WAW
Read after write	Write after read	Write after write
True dependency	Anti-dependency	Output dependency
Cannot simply reorder	Can often be fixed by renaming	Can often be fixed by renaming
Example: use loaded value	overwrite before old read	two writes same reg

9. Common Mistakes

- Confusing scheduling with CPU scheduling by OS.
- Reordering across memory operations without alias safety.
- Ignoring register pressure.
- Treating all dependencies as true dependencies.
- Assuming modern CPUs make compiler scheduling useless.

10. Edge Cases / Special Cases

- Volatile operations preserve source order.
- Exceptions can prevent moving faulting operations.
- Function calls may have unknown side effects.
- Memory fences block reordering.
- Predicated instructions change scheduling possibilities.

11. How to Explain in Interview

"Instruction scheduling reorders independent machine instructions to reduce pipeline stalls. It uses dependency and latency information, while preserving true data dependencies, memory ordering, calls, and side effects."

12. Quick Revision Notes

- Key definition: safe instruction reordering for performance.
- Important point: hide latency.
- Common comparison: RAW vs WAR vs WAW.
- Must remember: scheduling can increase register pressure.
- Trap: not the same as OS process scheduling.

13. Practice Tasks

- Reorder a sequence to hide load latency.
- Build a dependency DAG for five instructions.

- Identify RAW, WAR, and WAW hazards.
- Explain why a volatile load cannot move.
- Schedule a tiny loop manually.

14. Final Cheat Sheet

- **Core definition:** Reorder instructions to reduce stalls.
 - **Why it matters:** Improves pipeline utilization.
 - **Most asked:** dependencies, list scheduling, latency, register pressure.
 - **Common comparison:** scheduling vs selection, RAW/WAR/WAW.
 - **One-line answer:** "Instruction scheduling keeps the CPU busy by moving independent instructions into otherwise idle cycles."
-

9. Calling Conventions

1. Overview

Calling conventions are rules for how functions call each other at machine-code level.

- **Definition:** ABI rules for passing arguments, returning values, saving registers, using the stack, and managing call frames.
- **Why it matters:** Independently compiled functions must agree on the same rules.
- **Where used:** Compilers, linkers, OS interfaces, foreign function interfaces, debuggers.
- **Why interviewers ask:** It tests stack-frame understanding and low-level function execution.

2. Core Idea

When one function calls another, both sides need a contract. The caller passes arguments and expects a return value. The callee may use registers but must preserve required state.

Analogy: A meeting protocol. Everyone agrees who brings documents, who records notes, and who cleans the room afterward.

Small example:

```
caller:
put args in registers/stack
CALL function
read return value

callee:
create stack frame
save callee-saved registers if used
compute result
restore saved registers
RET
```

Step-by-step:

1. Caller places arguments according to ABI.
2. Caller saves caller-saved registers if needed.
3. **CALL** stores return address and jumps.
4. Callee creates stack frame.
5. Callee executes and places return value.
6. Callee restores required registers and returns.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Argument passing	Where parameters go	Caller/callee agreement	registers then stack	ABI knowledge
Return value	Where result is placed	Caller retrieves result	return register	Function result
Caller-saved	Caller must save if needed	Calls may clobber	temp registers	Register allocation
Callee-saved	Callee restores if used	Preserves caller state	saved registers	Prologue/epilogue
Stack frame	Function's stack area	Locals, spills, return info	frame pointer offsets	Recursion/debugging
Prologue/epilogue	Entry/exit setup code	Maintains ABI	push/pop frame	Assembly pattern

4. Real-World Example

When Python calls a C extension, the boundary depends on platform ABI rules. The C compiler, Python runtime, linker, and OS must agree on argument passing, stack alignment, and return-value handling.

5. Diagrams / Mental Models

Typical stack frame:

high addresses

+-----+

| caller frame |

+-----+

| return address |

+-----+

| saved frame ptr |

+-----+

| callee-saved regs|

+-----+

| local variables |

+-----+

| spill slots |

```
+-----+  
low addresses
```

6. Common Interview Questions

1. What is a calling convention?

Rules for machine-level function calls. Expected: args, return, registers, stack. Mistake: source function syntax.

2. Why are calling conventions needed?

So separately compiled code interoperates. Expected: ABI compatibility. Mistake: only compiler convenience.

3. How are arguments passed?

Usually first few in registers, remaining on stack, depending on ABI. Expected: ABI-specific. Mistake: always stack.

4. Where is return value stored?

Usually a designated return register or memory for large returns. Expected: ABI rule. Mistake: arbitrary register.

5. What are caller-saved registers?

Registers the caller must save if it needs them after a call. Expected: may be clobbered. Mistake: callee saves them.

6. What are callee-saved registers?

Registers the callee must restore if it uses them. Expected: preserved across call. Mistake: caller restores them.

7. What is a stack frame?

Per-call stack storage for return info, saved registers, locals, spills. Expected: recursion support. Mistake: global memory area.

8. What is function prologue?

Entry code that sets up frame and saves registers. Expected: stack/frame pointer. Mistake: first source statement.

9. What is function epilogue?

Exit code restoring frame/registers and returning. Expected: reverse of prologue. Mistake: source `return` only.

10. How does recursion work at machine level?

Each call gets its own stack frame. Expected: separate locals/return addresses. Mistake: one copy of locals.

7. Deep-Dive Questions

1. How do variadic functions affect calling convention?

They need rules that let callee access unknown number/types of arguments, often involving stack/register save areas.

2. How are large structs returned?

Often through hidden pointer argument to caller-allocated storage.

3. Why does stack alignment matter before calls?

ABI and SIMD instructions may require aligned stack for correct/fast access.

4. What is tail-call optimization's calling convention challenge?

Caller frame can be reused only if arguments and ABI constraints allow safe jump instead of call.

5. How does FFI depend on calling convention?

Languages must agree on ABI or arguments/registers will be misinterpreted.

8. Comparison Tables

Caller-Saved	Callee-Saved
Caller saves if needed	Callee saves if used
Good for short-lived temporaries	Good for values live across calls
Clobbered by calls	Preserved across calls
More caller work around calls	More callee prologue/epilogue work

Stack Arguments	Register Arguments
Slower access	Faster access
Supports many args	Limited count
Simple ABI fallback	Common for first args
Uses memory	Uses register file

9. Common Mistakes

- Mixing up caller-saved and callee-saved.
- Assuming every argument is passed on stack.
- Forgetting return address.
- Ignoring stack alignment.
- Thinking recursion requires special compiler magic beyond stack frames.

10. Edge Cases / Special Cases

- Variadic functions.
- Struct return values.
- Tail calls.
- Exceptions/unwinding metadata.
- Interrupt handlers may use special conventions.

11. How to Explain in Interview

"A calling convention is the ABI contract for function calls: where arguments and return values go, which registers are preserved, how the stack frame is laid out, and how call/return happen."

12. Quick Revision Notes

- Key definition: machine-level function-call rules.
- Important point: caller and callee must agree.
- Common comparison: caller-saved vs callee-saved.
- Must remember: stack frame stores per-call state.
- Trap: ABI is not API.

13. Practice Tasks

- Draw stack frames for recursive factorial.
- Mark caller-saved and callee-saved responsibilities in a call.
- Translate a simple function call into pseudo-assembly.
- Explain how arguments 1 through 8 might be passed on a register-first ABI.
- Trace prologue and epilogue instructions.

14. Final Cheat Sheet

- **Core definition:** ABI rules for calling functions.
 - **Why it matters:** Enables separately compiled code to work together.
 - **Most asked:** stack frame, caller/callee-saved, argument passing.
 - **Common comparison:** caller-saved vs callee-saved.
 - **One-line answer:** "Calling conventions are the low-level contracts that make function calls work across compiled code."
-

10. JIT Optimization

1. Overview

JIT optimization improves code at runtime using information observed while the program executes.

- **Definition:** Just-In-Time compiler optimizations performed during execution, often after profiling hot code.
- **Why it matters:** Runtime information can enable optimizations impossible or unsafe at static compile time.
- **Where used:** JVM, .NET CLR, V8 JavaScript engine, PyPy, database query engines, regex engines.
- **Why interviewers ask:** It tests modern runtime knowledge: profiling, speculation, deoptimization, and tiered compilation.

2. Core Idea

A JIT starts with quick code, observes what runs often, then recompiles hot parts with aggressive assumptions.

Analogy: A delivery driver first follows normal roads, then learns frequent routes and takes shortcuts. If a road closes, they fall back.

Small example:

```
function add(a, b) {  
  return a + b;  
}
```

If profiling shows `a` and `b` are usually integers, a JavaScript JIT may generate fast integer-add code. If strings appear later, it deoptimizes to generic code.

Step-by-step:

- 1. Interpret or baseline-compile code quickly.
- 2. Collect profiling data.
- 3. Identify hot functions/loops.
- 4. Optimize using runtime assumptions.
- 5. Execute optimized machine code.
- 6. Deoptimize if assumptions break.

3. Important Subtopics

Subtopic	Meaning	Why It Matters	Example	Interview Angle
Hot code	Frequently executed code	Worth optimizing	loop run 1M times	Profiling
Tiered compilation	Multiple compilation levels	Balances startup and speed	interpreter -> baseline -> optimizing	Runtime strategy
Speculation	Optimize assuming common case	Fast dynamic languages	integer-only add	Guards
Deoptimization	Return to safer code	Correctness when assumptions fail	integer add sees string	Runtime safety
Inline caching	Cache method/property lookup	Speeds dynamic dispatch	JS property access	Hidden classes
Inlining	Replace call with body	Removes call overhead	small getter	Enables further opts
Escape analysis	Check if object escapes scope	Stack allocation/removal	temporary object	Allocation reduction

4. Real-World Example

In Chrome's V8 engine, JavaScript property access can be optimized using hidden classes and inline caches. If objects keep the same shape, lookup becomes fast. If many shapes appear, the JIT may use a more generic path.

5. Diagrams / Mental Models

Runtime execution:

```

source/bytecode
  |
  v
interpreter or baseline JIT
  |
  v
profiling: hot? types stable?
  |
  v
optimizing JIT
  |
  v
guards pass? -> fast code
guards fail? -> deopt to safe code

```

6. Common Interview Questions

1. What is JIT compilation?

Compiling code to machine code during program execution. Expected: runtime compilation. Mistake: same as interpretation.

2. Why can JIT be faster than static compilation for dynamic languages?

It uses runtime type/profile information. Expected: speculation and guards. Mistake: JIT always faster.

3. What is hot code?

Code executed frequently enough to justify optimization. Expected: profiling threshold. Mistake: code with high memory usage.

4. What is tiered compilation?

Multiple execution tiers balancing startup and peak performance. Expected: interpreter/baseline/optimized. Mistake: one compiler only.

5. What is speculative optimization?

Optimizing based on likely runtime behavior with guards. Expected: assumptions. Mistake: unsafe guessing.

6. What is deoptimization?

Falling back from optimized code when assumptions fail. Expected: correctness mechanism. Mistake: just crash/stop.

7. What is inline caching?

Caching dynamic lookup results at call/access sites. Expected: speed property/method lookup. Mistake: general CPU cache.

8. Why does JIT have startup overhead?

It spends runtime compiling and profiling. Expected: warm-up cost. Mistake: no compile cost.

9. What is inlining in JIT?

Replacing a call with callee body. Expected: removes call overhead and exposes optimizations. Mistake: same as macro text replacement always.

10. Why can optimized JIT code become invalid?

Runtime behavior can violate assumptions, such as type or object-shape changes. Expected: guards/deopt. Mistake: compiled code never changes.

7. Deep-Dive Questions

1. What is on-stack replacement (OSR)?

Switching from interpreted/baseline code to optimized code while already inside a running loop.

2. How does deoptimization reconstruct interpreter state?

Optimized code keeps metadata mapping machine state back to logical frames and variables.

3. What is polymorphic inline cache?

A cache that handles a small number of observed receiver shapes/types at one call site.

4. Why does JIT need guards?

Guards check speculative assumptions before using fast specialized code.

5. How does escape analysis help JITs?

If an object does not escape, allocation can be removed or placed on stack-like storage.

8. Comparison Tables

Interpreter	JIT Compiler	AOT Compiler
Executes code directly	Compiles during runtime	Compiles before runtime
Fast startup	Medium startup	No runtime compile cost
Lower peak speed	High peak speed	High predictable speed
Easy debugging	Needs deopt metadata	Good for native binaries
Baseline JIT	Optimizing JIT	
Compiles quickly	Compiles slower	
Less optimized	More optimized	
Good startup	Good hot-code speed	
Simple assumptions	Profile-guided speculation	

9. Common Mistakes

- Saying JIT is always faster than AOT.
- Ignoring warm-up time.
- Confusing inline cache with CPU cache.
- Forgetting deoptimization is required for correctness.
- Assuming speculative optimization means unsafe optimization.

10. Edge Cases / Special Cases

- Cold code may never be optimized.
- Megamorphic call sites can defeat inline caches.
- Security sandboxes restrict generated executable memory.
- JIT compilation can interact with garbage collection safepoints.
- Mobile or embedded environments may limit JIT due to memory/security policy.

11. How to Explain in Interview

"JIT optimization compiles and optimizes code at runtime. It profiles execution, optimizes hot paths using assumptions like stable types, guards those assumptions, and deoptimizes back to safe code if they fail."

12. Quick Revision Notes

- Key definition: runtime compilation and optimization.
- Important point: profile-guided and speculative.
- Common comparison: interpreter vs JIT vs AOT.
- Must remember: deoptimization preserves correctness.
- Trap: JIT has warm-up cost.

13. Practice Tasks

- Trace how a dynamic `add(a,b)` function can be optimized for integers.
- Explain why a polymorphic call site is slower than monomorphic.
- Draw a tiered compilation pipeline.
- Identify guards needed for optimized property access.
- Compare startup time and peak speed for interpreter, JIT, and AOT.

14. Final Cheat Sheet

- **Core definition:** Runtime compilation and optimization using profile data.
- **Why it matters:** Speeds hot paths in dynamic/runtime-managed systems.
- **Most asked:** hot code, tiering, speculation, guards, deoptimization.
- **Common comparison:** interpreter vs JIT vs AOT.
- **One-line answer:** "A JIT watches runtime behavior, optimizes hot code for the common case, and falls back safely when assumptions break."

Whole-Topic Quick Comparison

Topic	Main Question It Answers	Key Interview Hook
Instruction selection	Which target instructions should implement this IR?	ISA patterns and costs
Register allocation	Which values should stay in registers?	Liveness and interference
Register spilling	What happens when registers run out?	Stack slots and spill cost

Topic	Main Question It Answers	Key Interview Hook
Addressing modes	How does an instruction find operands?	Arrays, pointers, fields
Target machine model	What machine rules must codegen follow?	ISA, ABI, registers, layout
Basic code generation	How do we emit correct assembly from IR?	Descriptors and basic blocks
Graph-coloring allocation	How can register allocation use graph theory?	K-coloring interference graphs
Instruction scheduling	How should instructions be ordered?	Pipeline stalls and dependencies
Calling conventions	How do functions call each other in machine code?	Stack frames and saved registers
JIT optimization	How does runtime optimization work?	Hot code, speculation, deopt

Final Placement Cheat Sheet

Topic	One-Line Interview Answer
Instruction selection	Choose target instructions that implement IR efficiently.
Register allocation	Assign live values to limited CPU registers.
Register spilling	Store less useful live values in memory when registers run out.
Addressing modes	Define how instructions locate operands in registers or memory.
Target machine model	Describe the CPU, ABI, data layout, registers, and costs.
Basic code generation	Translate IR to assembly while tracking value locations.
Graph-coloring allocation	Color the interference graph using registers as colors.
Instruction scheduling	Reorder independent instructions to reduce pipeline stalls.
Calling conventions	Define machine-level rules for calls, returns, registers, and stack.
JIT optimization	Optimize hot code at runtime using profiling and guarded assumptions.